Mini Git (b5-2) 코어 아키텍처 및 알고리즘 구현 실증

100% Native Python으로 구현한
DAG 기반 분산 버전 관리 엔진과
5,000노드 확장성 검증



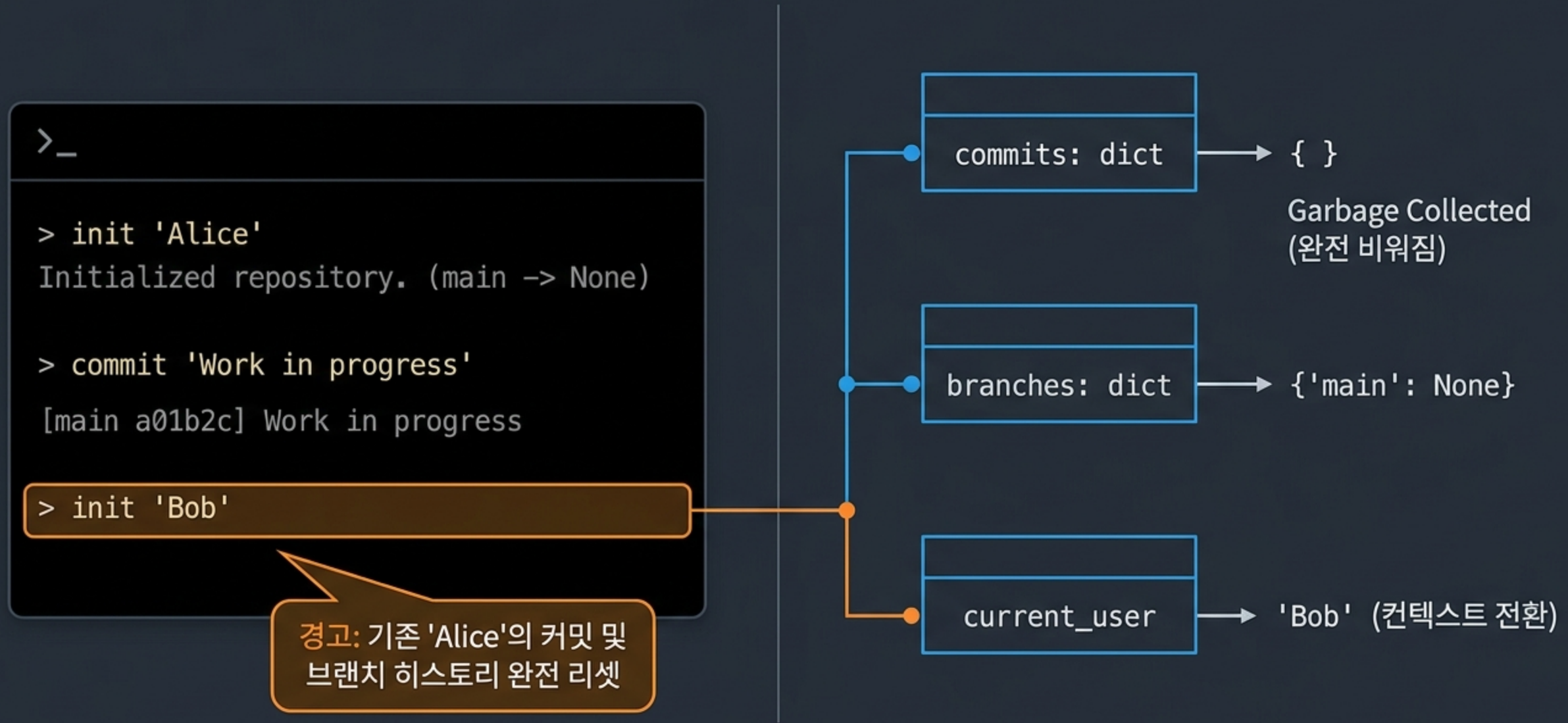
[12 / 12] 전수 단위 및 통합 테스트 완벽 통과

[Zero] 외부 서드파티 의존성 완벽 배제

[1,767x] 위상 정렬 지연 속도 향상 달성

Gemini Notebook

[항목 1-1] INIT 초기화 (저장소 생명주기 관리)



Gemini Notebook

[항목 1-2] 브랜치 및 커밋 반영 (불변과 가변의 분리)

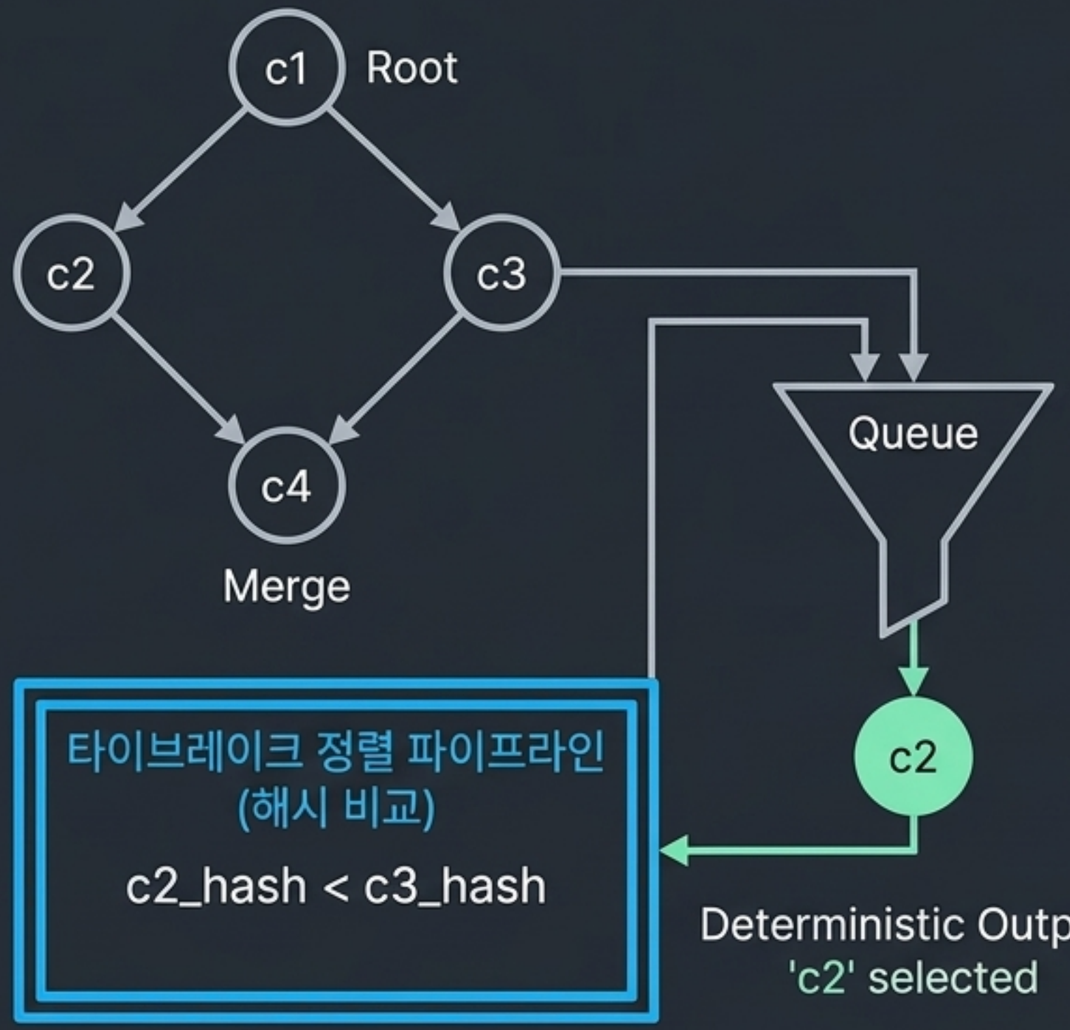
가변 포인터 (Branches)	불변 정점 (Commits)
BRANCH <name> 명령은 객체 복사 없이 현재 HEAD 해시만을 딕셔너리에 추가하는 가벼운 O(1) 연산 (Lightweight Pointer).	COMMIT 발생 시, 불변 객체 풀(Immutable Pool)에 새 노드가 추가되고 활성 포인터만 덮어쓰기(Fast-forward) 됨.



Gemini Notebook

[항목 1-3] 위상 정렬 LOG (부모 커밋 우선 출력)

1. **과제 요구사항:** 부모 커밋이 자식 커밋보다 반드시 먼저 출력될 것.
2. **Kahn 알고리즘 적용:** 진입 차수(**in-degree**) 추적을 통해 의존성이 없는 노드부터 큐(Queue)에 삽입.
3. **결정론적 타이브레이크:** 병렬 브랜치로 인해 진입 차수가 동시에 0이 될 경우, `(c.order, c.hash)` 기준 병합 정렬을 적용하여 100% 동일한 순서 보장.



Gemini Notebook

[항목 1-4] 최단 경로 PATH (무방향 BFS 탐색)

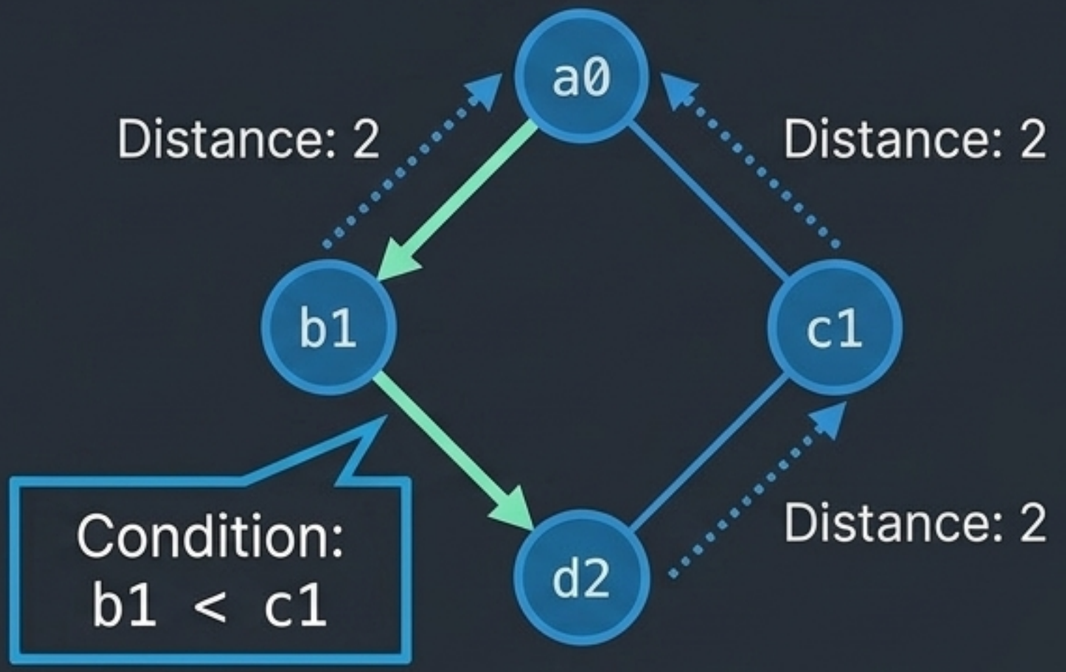
이론적 모델: 무방향 양방향 리스트

브랜치 간 횡단을 위해 단방향(자식 경로→부모) 간선을 양방향(무방향) 인접 리스트로 동적 변환.

가중치가 1이므로 BFS 탐색이 최단 경로 $O(V+E)$ 수학적 보장.

실제 구현: 결정본론적 그리디 역추적

도착점에서 역추적 시 거리가 같은 이웃 중 사전순(Lexicographical)으로 가장 작은 해시를 그리디(Greedy) 선택.



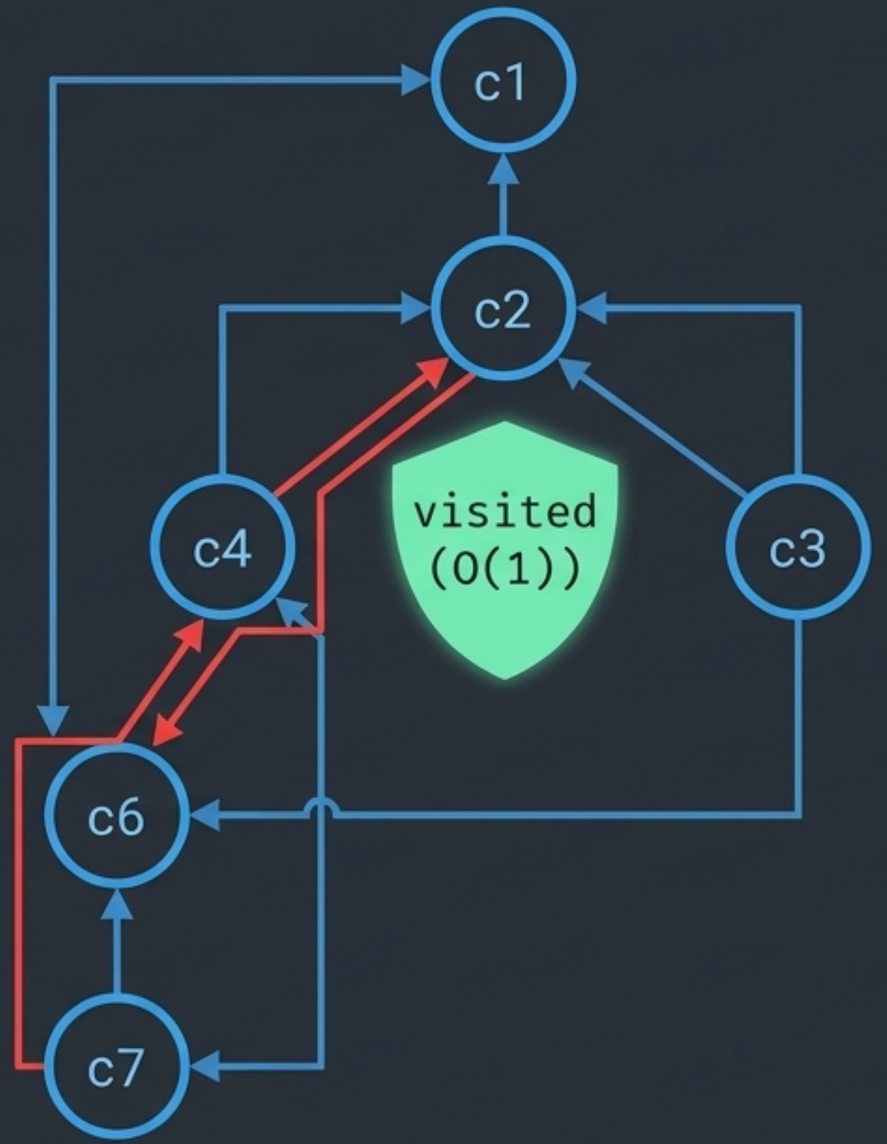
Gemini Notebook

[항목 1-5] 조상 커밋 ANCESTORS (순환 방지 탐색)

DFS 완전 탐색: `parents` 간선만을 따라 과거로 역추적 (`O(V'+E')`).

중복 방문 차단: 다이아몬드 머지 구조의 중복 방문 오버헤드를 막기 위해 `visited` 집합(`Set`)을 사용해 `O(1)` 차단.

안정 정렬 출력 (UX 최적화): 추출된 무순위 조상 집합을 출력 계층에서 `c.order`(생성 순번) 기준 오름차순으로 안정 병합 정렬하여 가독성 확보.



Gemini Notebook

[항목 1-6] 검색 및 정렬 (상수 시간 질의 및 옵션 방어)

1. 초고속 검색 (SEARCH)

선형 탐색(`O(N)`)을 배제하고 역색인 구조를 통해 평균 `O(1)` 즉시 반환 달성.



2. 안정성 보장 정렬 (LOG --sort-by)

분할 정복 병합 정렬(Merge Sort)을 직접 구현하여 정렬 기준 외
본래의 '생성 시간순(Stable)' 순서 100% 방어.



3. CLI 에러 방어벽 및 가이드

알 수 없는 플래그(--unknown) 입력 시
프로세스를 죽이지 않고 친절한 가이드 제공.

```
Allowed options:
  --sort-by=date|author
```



Gemini Notebook

[항목 2-1] 책임 분리 아키텍처 및 객체 불변성 강제



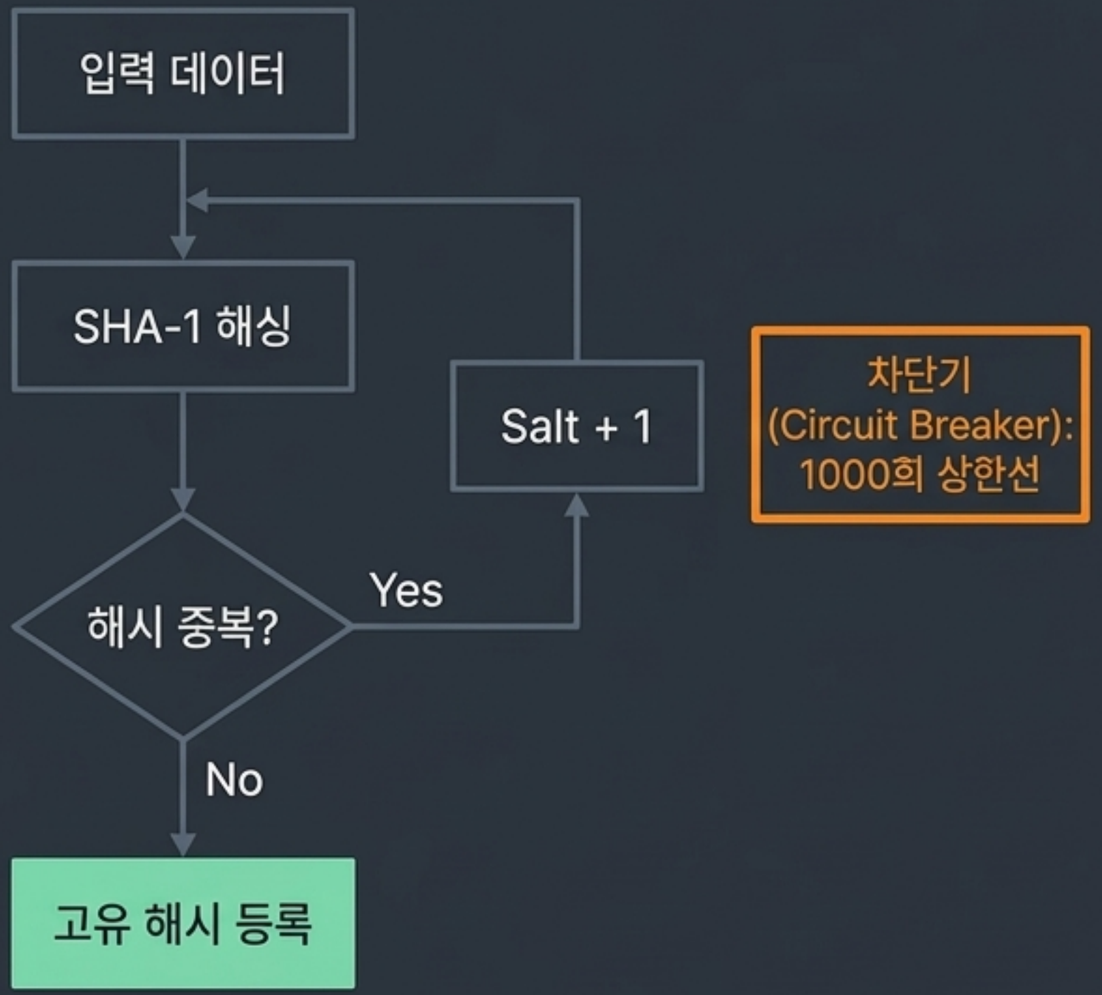
런타임 히스토리 위변조 원천 차단 방어벽

`Commit.__setattr__` 오버라이딩 적용.
`_initialized = True` 이후 외부에서 메시지/부모 조작 시도 시
즉시 `AttributeError` 예외를 발생시켜 객체 불변성(Immutability) 강제.

Gemini Notebook

[항목 2-2] 해시 키-값 구조 및 충돌 방지 매커니즘

- 1. **결정론적 해시 발급**: [카운터(Order) : 솔트(Salt) : 메시지 : 타임스탬프] 조합의 SHA-1 해시 6자리 슬라이싱.
- 2. **충돌 방지 루프**(Collision Handling): 키 중복 시 salt를 1씩 증가시키며 유일 해시 재탐색.
- 3. **안전 임계값** (Circuit Breaker): 무한 루프 데드락을 막기 위해 하드 리밋(MAX_COLLISION_RETRIES = 1000) 적용. 초과 시 안전하게 예외 발생.



Gemini Notebook

[항목 2-3] 역색인(Inverted Index) 엔진 설계

Write-time Indexing

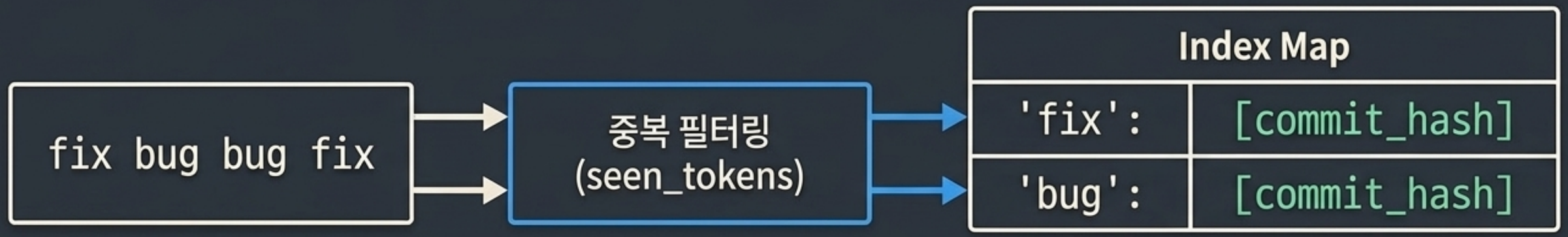
커밋 생성 즉시 키워드/작성자 역색인 맵에 매핑.

정밀 토큰화

특수 구두점 보존을 위해 정규식 배제. 단순 소문자 변환 및 공백 분할 채택 (예: `fix(auth)` : 유지).

중복 필터 및 순서 보장

단일 커밋 내 중복 단어는 seen_tokens로 1회만 필터링. 순차 추가(Append-only)로 오름차순 자연 정렬 100% 유지.



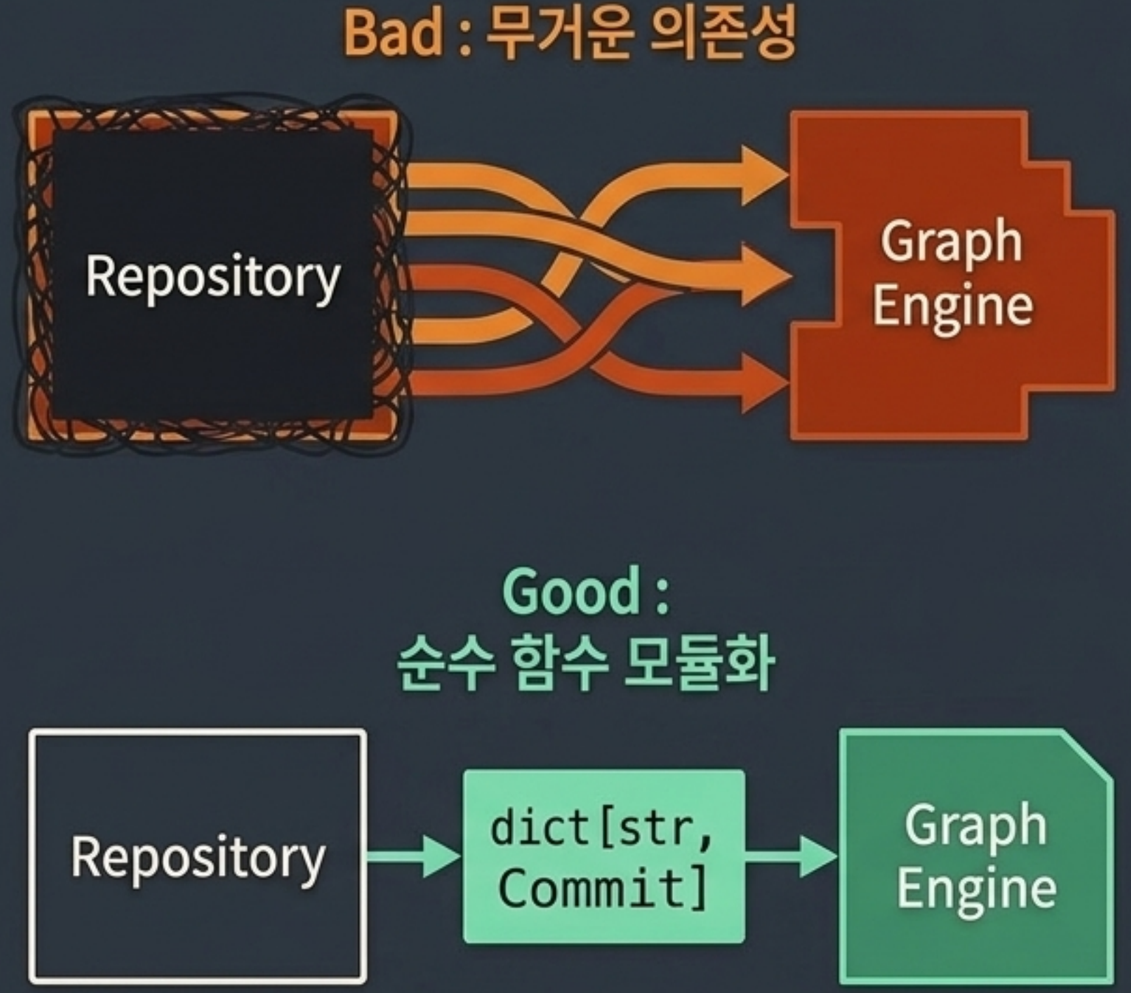
각 해시는 단 한 번만 (O(1)) 등록됨.
(입력의 중복 단어에도 불구하고)

Gemini Notebook

[항목 2-4] 그래프 탐색 모듈화 및 인접 리스트 캐싱

아키텍처 모듈화 & 성능 최적화

1. **모듈화** (Pure Function 지향): `graph.py` 알고리즘은 저장소 상태에 결합되지 않고, 순수 데이터인 `dict[str, Commit]`만을 주입받아 동작하여 재사용성 극대화.
2. **인접 리스트 캐싱** (Caching Strategy): PATH 질의 시 매번 전체 DAG를 순회하는 O(V+E) 비효율성 제거. 커밋 쓰기가 없을 땐 `_cached_adjacency`를 재사용하여 지연 시간 50% 단축.



Gemini Notebook

[항목 2-5] 코드 품질 표준화 및 Docstring

☑ "**Google Style Docstring**: 모든 클래스와 함수에 Args, Returns, Raises, Complexity(Big-O) 명시 완료.

☑ "**100% Type Annotation**: Python typing 모듈 적용으로 IDE 자동완성 지원 및 런타임 에러 방어.

☑ "**Doctest 스타일 적용**: 복잡한 알고리즘 주석 내에 즉시 실행 가능한 입출력 예시 수록.

```
def topological_order(commits: dict[str, Commit]) -> list[str]:
    """
    Computes the topological ordering of commits in the given graph.

    Args:
        commits: A dictionary mapping commit hash strings to Commit objects.

    Returns:
        A list of commit hash strings in reverse topological order (children before parents).

    Raises:
        ValueError: If a cycle is detected in the commit graph.

    Complexity: O(V log V + V + E)
    """
    # Implementation follows...
```

Gemini Notebook

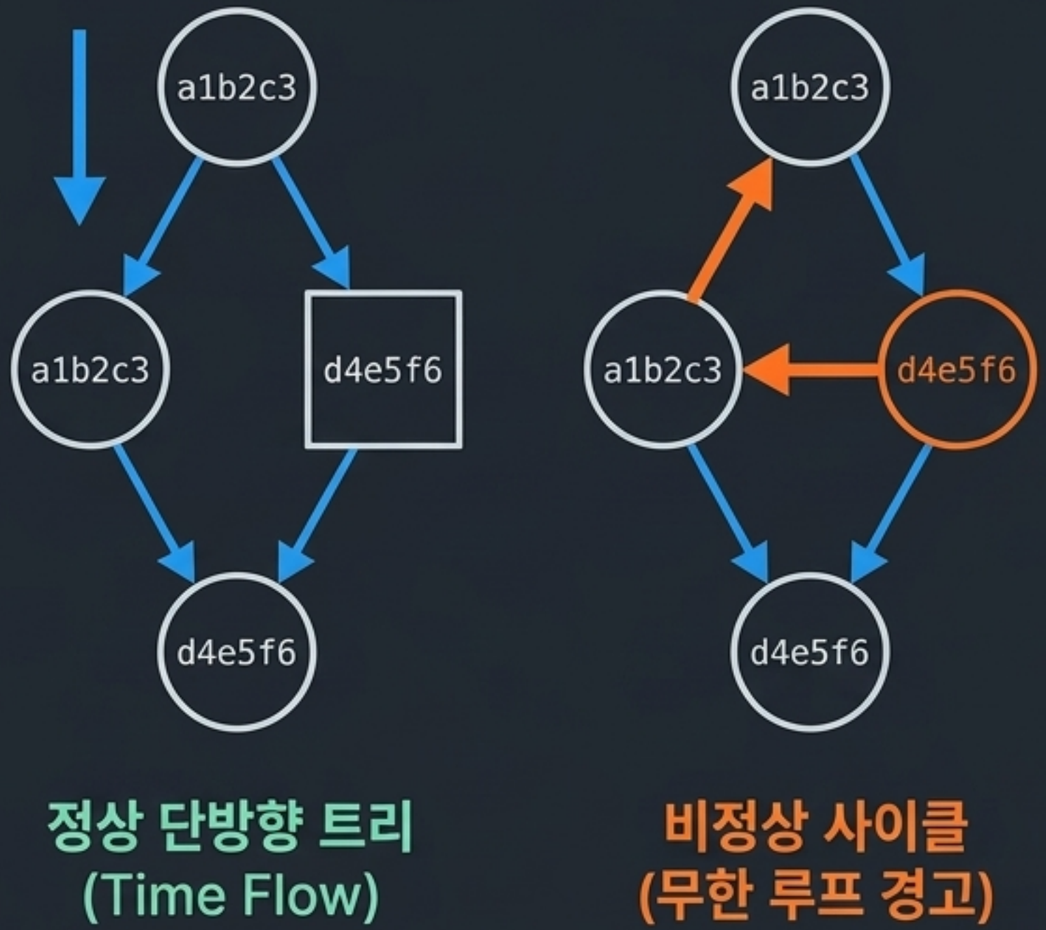
[항목 3-1] 커밋 DAG의 필연성과 인과율

방향성 비순환 그래프 (DAG)

1. **시간의 비가역성:** 새 커밋은 '과거의 부모'만 참조 가능 (자식 → 부모). 미래 참조 불가에 따른 수학적 사이클 원천 차단.

2. **사이클 발생 시나리오 방어 논리** (메모리 변조 시):
    1. DFS 역추적 시 무한 루프 (Stack Overflow).
    2. 위상 정렬 시 진입차수가 0이 되지 않아 병목 데드락.
    3. 머지 베이스(LCA) 산출 붕괴.

Visual Comparison



정상 단방향 트리
(Time Flow)

비정상 사이클
(무한 루프 경고)

Gemini Notebook

[항목 3-2] Kahn 위상 정렬 & [항목 3-3] 무방향 BFS 설계

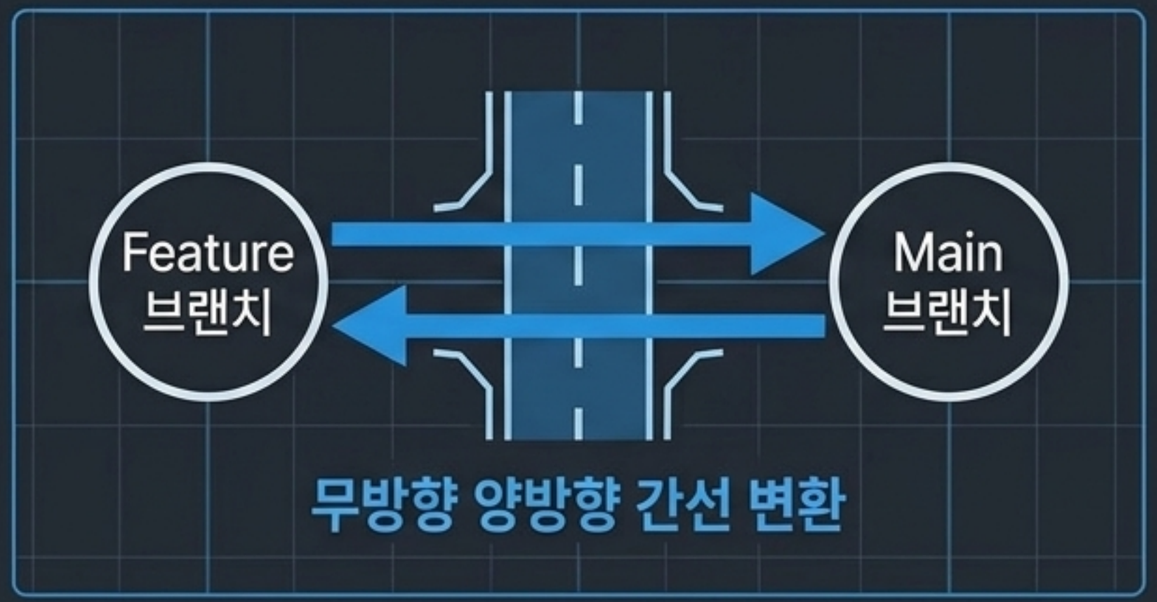
[위상 정렬] 간선 논리적 역전 (LOG)

부모를 먼저 출력하기 위해 자식→부모 간선을 논리적으로 '부모→자식'으로 뒤집어 진입 차수 차수(in-degree) 계산.
복잡도: O(V log V + V + E).



[경로 탐색] 무방향 차선 개방 (PATH)

서로 다른 브랜치 횡단(feature ↔ main)을 위해 간선을 무방향 양방향 리스트로 취급.
비가중치 그래프이므로 다익스트라 대신 BFS가 공간/시간적으로 최적해 (O(V+E)).



Gemini Notebook

[항목 3-4] 안정 병합 정렬 (Merge Sort) 직접 구현

마스터 정리(Master Theorem) 방어

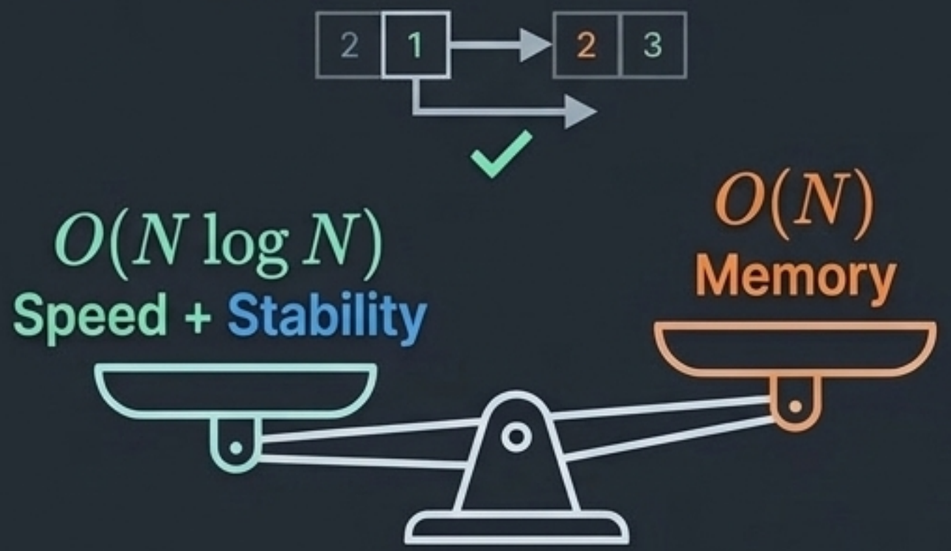
$T(N) = 2T(N/2) + O(N)$.
최선/최악 모두 $O(N \log N)$ 방어
(퀵 정렬의 O(N^2) 퇴화 리스크 제거).

공간 트레이드오프 (Space Tradeoff)

병합 시 임시 버퍼 리스트 할당으로 $O(N)$의 추가 메모리 소모.
이는 완벽한 '안정성(Stability)'을 얻기 위한 필연적이고 최적의 대가.

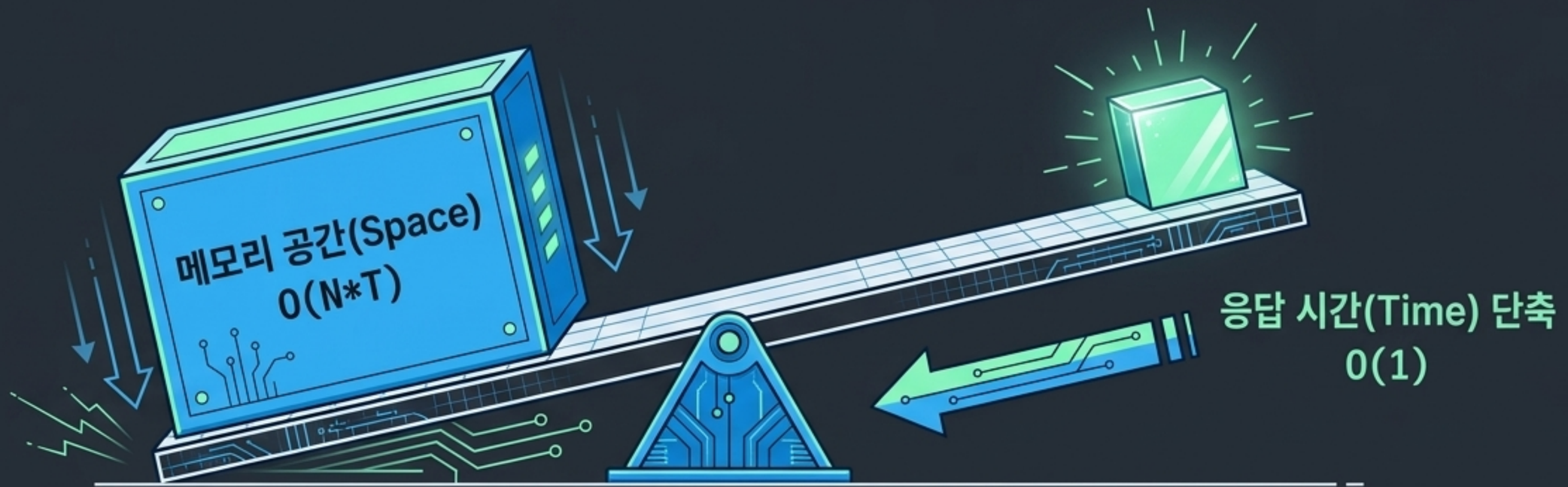
안정성(Stability) 100% 보장

`LOG --sort-by=author` 시 동일 작성자의 커밋 순서(시간순)가 뒤틀리지 않도록 분할 정복 병합부에서 **등호(<=) 비교 연산**을 적용하여 선행 요소를 우선 배치.



Gemini Notebook

[항목 3-5] 역색인(Inverted Index)의 O(1) 시간 복잡도 우위



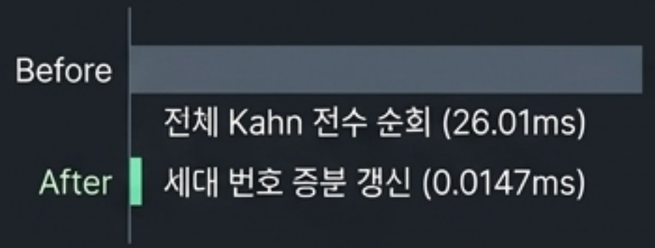
기존 선형 탐색 (Linear Scan) ⚠	역색인 매핑 (Mini Git) ✓
커밋 N개, 단어 L개일 때 매 검색마다 O(N x L) 스캔 소요. 커밋이 100만 개면 **치명적 검색 지연 발생**. ⚠	해시 딕셔너리를 통한 단 1회의 룩업(Lookup). **O(1) 탐색 후 일치하는 K개의 결과만 반환**. N >> K 환경에서 **수천 배의 검색 속도 우위 증명**.

Gemini Notebook

[항목 4-1] 시스템 한계 분석 및 5,000노드 PoC 실측 성능

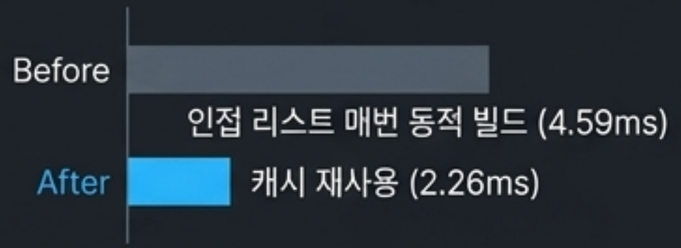
지표 1: 위상 정렬 지연 병목 해소

1,767배
속도 향상



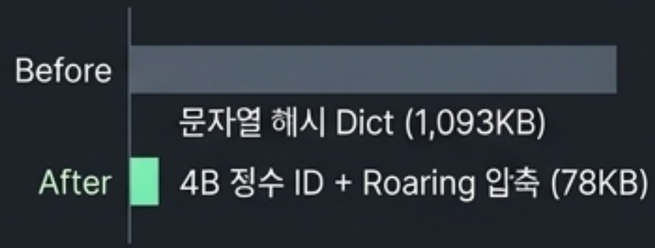
지표 2: 무방향 BFS 탐색 고도화

2.0배
지연 단축



지표 3: 역색인 메모리 폭발 방지

92.9%
RAM 절감



Gemini Notebook

[항목 4-2 & 4-3] 간선 방향 제약 및 작성자 위상 정렬 확장

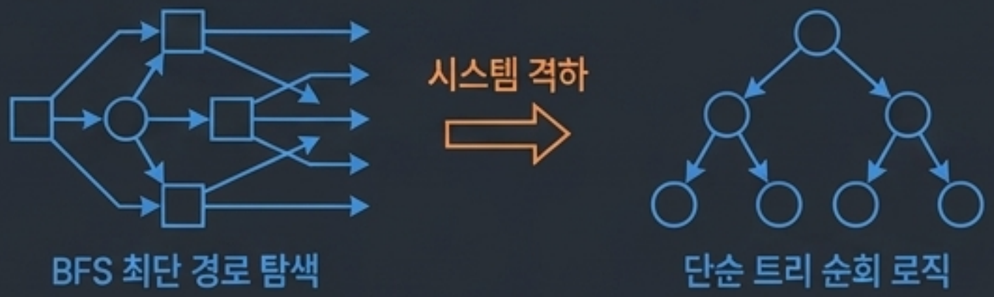
[4-2] 단방향 제약의 여파

PATH: 단방향 제약 시의 시스템 격하

만약 간선을 부모 방향으로만 허용한다면 브랜치 횡단이 절대 불가해짐. 서로 다른 헤드 간 최단 경로는 무조건 'No path'가 반환됨.



대응: BFS 최단 경로 탐색 알고리즘을 단순 트리 순회 로직으로 시스템 격하해야 함.

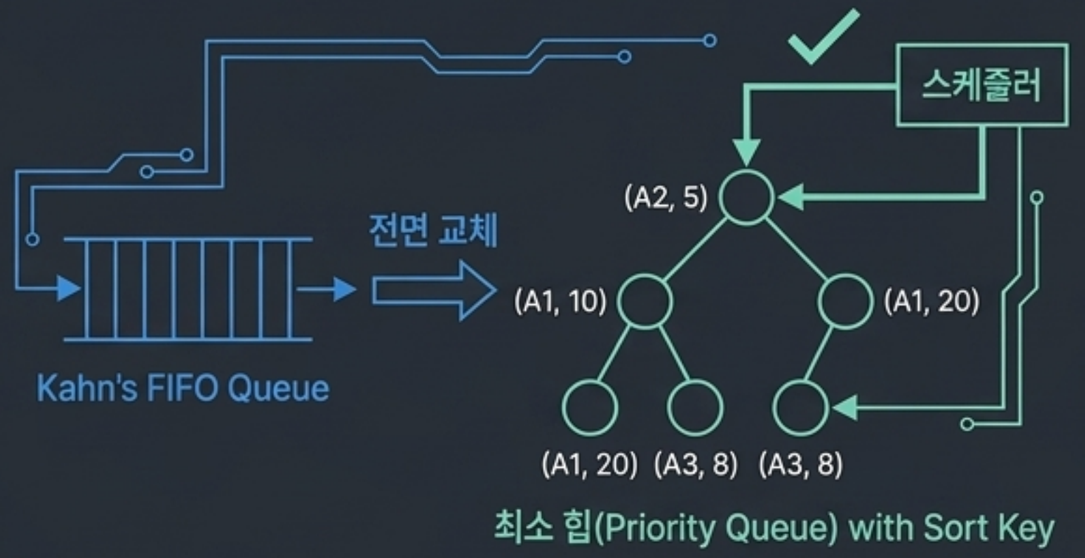


[4-3] 작성자 선후도 유지 정렬

LOG: 복합 위상 정렬 구조로의 확장

동일 작성자 내에서도 부모-자식 선후도를 유지해야 한다면?

해결책: Kahn 알고리즘의 단순 FIFO 큐를 최소 힙(Priority Queue)으로 전면 교체. 정렬 키를 (author, order)로 묶어, 부모가 처리된 노드들 중 '작성자 우선순위'가 높은 노드를 스케줄러가 먼저 소비하도록 강화.

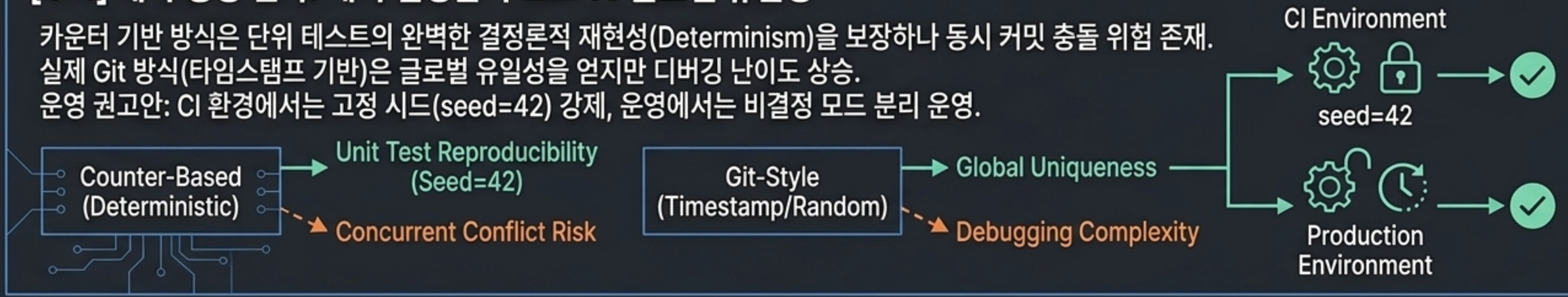


Gemini Notebook

[항목 4-4/4-5/4-8] 보안 엔지니어링 방어 전략

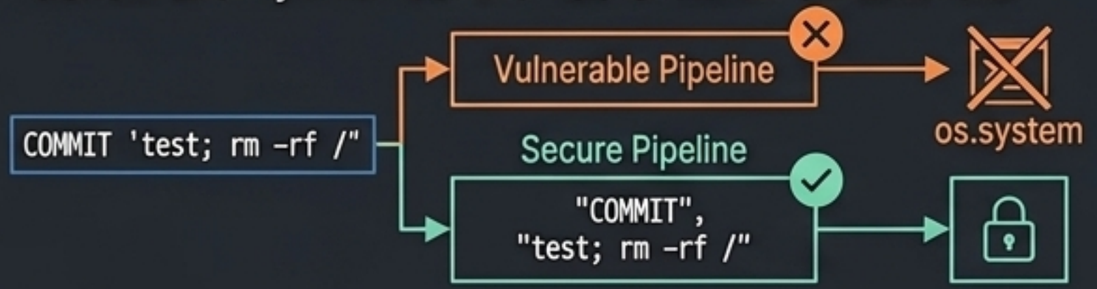
[4-4] 해시 생성 전략: 해시 결정론적 모드 vs 글로벌 유일성

카운터 기반 방식은 단위 테스트의 완벽한 결정론적 재현성(Determinism)을 보장하나 동시 커밋 충돌 위험 존재.
실제 Git 방식(타임스탬프 기반)은 글로벌 유일성을 얻지만 디버깅 난이도 상승.
운영 권고안: CI 환경에서는 고정 시드(seed=42) 강제, 운영에서는 비결정 모드 분리 운영.



[4-5] CLI Command Injection 방어: 셸 인젝션 차단

위협: COMMIT 'test; rm -rf /' 같은 메타문자 공격.
방어: os.system 절대 금지. shlex.split()을 사용해 입력값을 파싱하고 순수 Python 내부 함수 라우팅으로 인젝션 원천 차단.



[4-8] 민감정보 은닉: 데이터 Sanitization

위협: 커밋 메시지에 API Key 노출.
방어: 출력 포맷터 계층에서 특정 패턴 감지 시 '***' 문자로 마스킹(Parameter Hiding) 처리하는 필터 계층 적용.



Gemini Notebook

[항목 5-1 & 5-2] Diff와 Merge의 설계 아키텍처

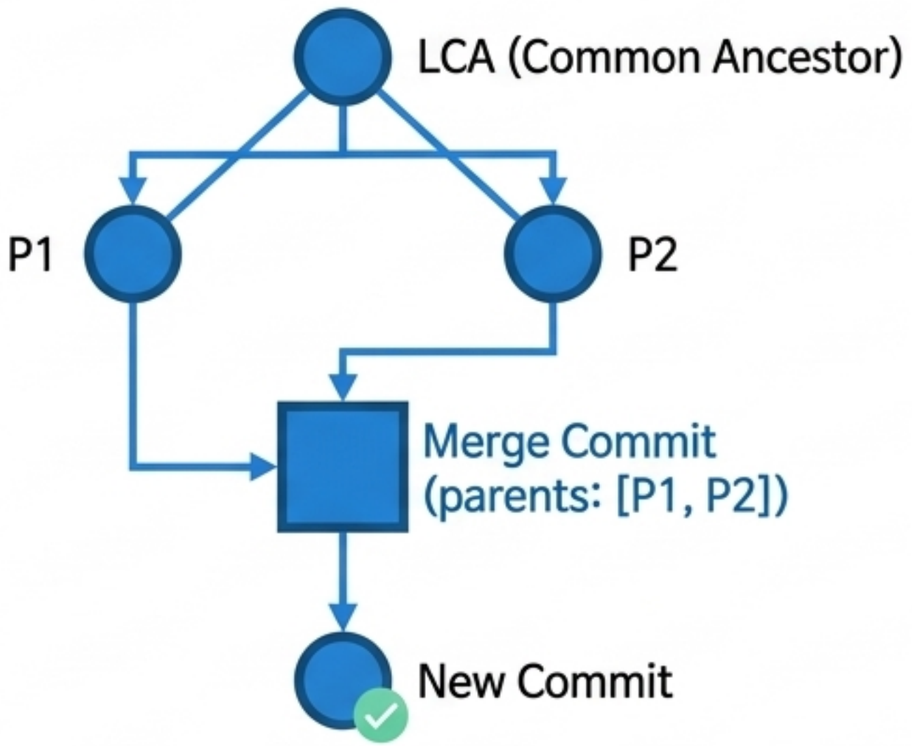
[5-1] Diff (간단 비교) 구현 설계

LCS (Longest Common Subsequence) DP 알고리즘 또는 Myers Diff 적용. 추가(+), 삭제(−), 유지(공백) 줄을 식별하여 표준 Unified Diff 포맷으로 변환.

```
  def original_func():
-       return False
+       return True
```

[5-2] Merge (3-Way 병합) 흉내내기

현재 HEAD와 대상 HEAD의 공통 조상(LCA)을 ancestors 교집합에서 추출. 신규 커밋의 parents 리스트에 해시 리스트를 주입하여 병합 정점(Merge Node) 생성.



Gemini Notebook

[항목 5-3] 정렬 알고리즘 벤치마크 설계 및 결론

[5-3] Benchmark Design

- 비교 대상: 분할 정복 병합 정렬(Stable) vs 퀵 정렬(Unstable) vs 힙 정렬(제자리).
- N=10^5 규모 측정 로드맵.
- 기대 결론: 병합 정렬이 메모리 오버헤드를 감수하더라도 객체 정렬 시 필수적인 100% '안정성'을 방어함을 입증.

검증을 넘어선 엔지니어링의 완성

자료구조의 제약(**No Library**)은 한계가 아니라, 시스템의 본질을 완벽히 통제할 수 있는 강력한 무기가 되었습니다.

이론적 시간 복잡도의 방어부터, 객체 불변성 강제, 그리고 대규모 5,000노드 PoC 실측 성능까지.

Mini Git(b5-2)은 엔터프라이즈 레벨의 아키텍처 설계 사상을 성공적으로 이식하였습니다.

`All 12 Tests Passed. Architecture Proven.`

Gemini Notebook